owenmurphy_murphy1
1430-2019-31
Program 18 URI: R+L=J

50

Program Output:	Solution:
Pass Documentation Check: prog18.cpp	
11 0 -2 38 -45 -65 6 0 0 34236654 39996 3780216 0 0 0 -6 -28236654 <P><hr><P>	11 0 -2 38 -45 -65 6 0 0 34236654 39996 3780216 0 0 0 -6 0 0 -28236654 -69046 -6737266
FAIL - URI: R+L=J	**PASS** - URI: R+L=J

50

Scanned with CamScanner

prog18.cpp
Upload: 10/14/2019 22:48:22

```
/*
Author: Owen Murphy
Assignment title: URI: R + L = J
Assignment Description: Write a program that tests cases and find the missing
   value in the algebra formula
Due Date: 10/10/2019
Date Created: 10/10/2019
Date Last Modified: 10/14/2019
*/

/*
 Data Abstraction:
    String declaration to read in the line
    input string stream to put line into a buffer
    integers R, L, and J to store the values of the integers
    more strings to read lines specific lines into 3 strings for later purpose\
    input string stream to read in the number
 Input:
    read in a line and also read into a string stream (input)
 Process:
    Read in each expression excludign the arithmetic expressions
    Once the missing variable is found blow the char in to a char variable
        and switch on the char finding the value of the missing variable
 Output:
    print the missing value to the screen and end with new line
 Assumptions:
    It is assumed that the string contains no spaces
    It is assumed that the test cases contain valid integers
    It is assumed that the format of the string does not change
 */

#include <iostream>
#include <sstream>
#include <string>
#include <cmath>
#include <iomanip>
#include <cctype>
#include <ios>
#include <cstdlib>
#include <vector>
using namespace std;

int main() {

    string str, str1, str2, str3;
    stringstream intStream;
    double r, l, j;
    char var;
    vector<double> number;
    vector<string> stringExp; // store each string expression from equation
    unsigned int i;
    unsigned int k;

    // Input
    while (getline(cin, str)) {

        stringExp.clear();
        intStream.clear();
        intStream.str(str); // only need initially to store string

        getline (intStream, str1, '+');
        getline (intStream, str2, '=');
        getline (intStream, str3);


```

Scanned with CamScanner

```
        stringExp.push_back(str1);
        stringExp.push_back(str2);
        stringExp.push_back(str3);

        //Process
        for (k = 0; k < stringExp.size(); ++k) {
            str = stringExp.at(k);
            intStream.clear(); // clear and reuse to store each expression
            for (i = 0; i < str.size(); ++i) {
                intStream.str(str);

                // Find the missing variable and integers present
                if (isdigit(str.at(i)) && (k == 0)) {
                    intStream >> r;
                }
                else if (k == 0) {
                    intStream >> var;
                }

                if (isdigit(str.at(i)) && (k == 1)) {
                    intStream >> l;
                }
                else if (k == 1) {
                    intStream >> var;
                }
                if (isdigit(str.at(i)) && (k == 2)) {
                    intStream >> j;
                }
                else if (k == 2) {
                    intStream >> var;
                }
            }
        }

         switch (var) {
             case 'R': case 'r':
                 r = j - l;
                 number.push_back(r);
                 break;
             case 'L': case 'l':
                 l = j - r;
                 number.push_back(l);
                 break;
             case 'J': case 'j':
                 j = l + r;
                 number.push_back(j);
                 break;
         }
    }

    // Output
    cout << fixed << setprecision(0);

    for (i = 0; i < number.size(); ++i) {
        cout << number.at(i) << endl;

    }

    return 0;
}
```

Scanned with CamScanner